

05 — Device-Side Completion (Hardware-Gated)

Revision: 1 **Parent:** 00_MASTER_INDEX.md **Prerequisites:** [HARDWARE]
RK3588 / Orange Pi 5 Max board physically attached or reachable over ADB/SSH.

Overview

The device-side code has TWO components: 1. **Android OTA agent** (submodules/ota-android-agent) + **update-engine bridge** (submodules/ota-update-engine-bridge) — Kotlin/JVM, proven in unit tests only. 2. **Linux/U-Boot ApplyPort** (server/internal/device/) — Go, scaffold only for real-partition path.

Both need real-hardware validation. The Android agent needs stress/chaos/bench/memory testing. The ApplyPort needs the slot-writer + driver loop completed.

E-01 [OPERATOR/HARDWARE] — Attach RK3588 Board

Source: Issues.md OTA-004

What to do:

1. Physically connect Orange Pi 5 Max (RK3588) board to a host machine via USB (ADB) and/or Ethernet (SSH).
2. Flash a known-good Android 15 AOSP system image to the board.
3. Verify reachability:

```
adb devices          # should list the device
adb shell getprop ro.product.board # should show rk3588_t or
                                similar
```

4. Verify A/B slot structure:

```
adb shell getprop ro.boot.slot_suffix # should show _a or _b
adb shell ls -la /dev/block/by-name/  # should show boot_a,
                                boot_b, system_a, system_b, etc.
```

5. Record the board's serial number, hardware ID for device registration.

Unblock condition (from Issues.md): “A physical RK3588 / Orange Pi 5 Max board reachable over ADB/SSH (or physically attached) for flashing.”

E-02 [HARDWARE/AGENT] — RK3588 Tier-3 On-Silicon A/B Apply

Effort: L **Source:** Issues.md OTA-004,
PRODUCTION_READINESS_PLAN.md §5 P2

What to validate:

1. **Control-plane reachability:** Device can reach the ota-server over network.
2. **Device registration:** Register the board with the server, obtain device token.
3. **Update check:** Device polls GET /api/v1/client/update, receives update offer.
4. **Artifact download:** Device downloads the OTA artifact (full payload or delta).
5. **Signature verification:** Device verifies Ed25519 signature (real crypto, not stub).
6. **Apply via update_engine:**
ReflectiveUpdateEngineApplyPort.applyVerified() calls
android.os.UpdateEngine.applyPayload() via reflection.
7. **Slot switch:** update_engine writes to inactive slot, sets slot priority.
8. **Reboot:** Device reboots to the new slot.
9. **Post-boot verification:**
 - ro.boot.slot_suffix shows the NEW slot
 - ro.boot.verifiedbootstate shows GREEN

- App version matches the update's version
10. **Telemetry:** Device reports success telemetry event back to server.

Evidence required:

- ADB logcat capture of the entire OTA cycle
 - Server logs showing the device lifecycle
 - Screenshots/screen recording of the device booting into the new slot
 - §11.4.153 video recording (window-scoped, vision-verified MP4)
-

E-03 [AGENT] — Complete Linux/U-Boot ApplyPort Slot-Writer + Driver Loop

Effort: M **Source:** PRODUCTION_READINESS_PLAN.md §2.4, OTA-038

Current state:

The ApplyPort is a SCAFFOLD (`server/internal/device/applyport.go:10-16`). The slot-writer (`slot.go`) has dd-based writing but the full apply loop (Steps a → f) is not wired. The CLI (`cmd/applyport/main.go`) returns a placeholder.

What to implement:

1. SlotWriter: Real implementation that:

- Detects current slot (A/B) from `/proc/cmdline` or `fw_printenv`
- Determines inactive slot
- Writes the artifact payload to the inactive slot's partition via dd
- Verifies the written hash matches the expected SHA-256

2. ApplyPort driver loop (Steps a → f):

- 1. Check for update via `GET /api/v1/client/update`
- 1. Download artifact to temp file
- 1. Verify Ed25519 signature
- 1. Write to inactive slot via SlotWriter
- 1. Set U-Boot env (`fw_setenv BOOT_ORDER` to try new slot first)
- 1. Set health-confirmation marker, reboot

3. **Health confirmation:** `health.go` — on next boot, check if we booted the new slot successfully. If yes, mark as healthy (clear bootcount). If no, U-Boot bootcount rollback triggers.
4. **CLI fix:** Remove the placeholder return in `cmd/applyport/main.go`.
5. **Tests:** `TestSlotWriterRealDD`, `TestApplyPortFullLoop`, `TestHealthConfirmation`, `TestApplyportCLI`

Dependency: Step G-07 (system image with OTA agent pre-installed) must provide the `fw_env.config` + RAUC bundle validated for the board.

E-04 [AGENT] — Android Agent: Stress/Chaos/Bench/Memory + Real-Device Tests

Effort: M **Source:** `PRODUCTION_READINESS_PLAN.md` §3, OTA-043

Current state:

Android agent has JVM unit tests only (47 tests in `:core`, some in `:android`). One `androidTest/` file (`OtaAgentOnDeviceTest.kt`) exists but cannot run without a system-UID/platform-signed device.

What to add:

1. **Stress tests:** Simulate rapid poll cycles, concurrent update checks, multiple devices hammering the server.
 2. **Chaos tests:** Network failures mid-download, corrupted artifacts, server returning 5xx, token expiry mid-cycle.
 3. **Benchmark tests:** Poll cycle duration, download throughput, verification time, apply time.
 4. **Memory tests:** No memory leaks across 1000+ poll cycles.
 5. **Real-device instrumentation:** Run `androidTest/` on the RK3588 board (requires platform-signed APK).
 6. **Missing unit tests** (from `DELTA_ANALYSIS` §4):
 - `BootStateObserver` — no host-runnable unit test
 - `ReflectiveUpdateEngineApplyPort` — no host-runnable unit test
 - `PollScheduler` — no unit test
 - `Dtos.fromWire` logic — exercised only indirectly
-

E-05 [AGENT] — Update-Engine Bridge: Real-Device Tests

Effort: M **Source:** PRODUCTION_READINESS_PLAN.md §3

Current state:

The bridge has JVM unit tests only (27 tests). The `androidTest/` directory is EMPTY (no instrumentation tests).

What to add:

- 1. Real-device instrumentation tests:** Verify that `ReflectiveUpdateEngine` actually:
 - Resolves `android.os.UpdateEngine` class on a real device
 - Calls `applyPayload()` successfully
 - Receives status callbacks (IDLE → DOWNLOADING → VERIFYING → FINISHED)
 - 2. BootStateObserver tests:** Verify `ro.boot.slot_suffix`, `ro.boot.verifiedbootstate`, `ro.boot.veritymode` are read correctly on real hardware.
 - 3. Stress/chaos:** What happens when `update_engine` fails mid-apply? Does the bridge propagate errors correctly?
 - 4. Benchmark:** `applyPayload` call latency, callback latency.
-

E-06 [AGENT] — Device Setup Wizard UI (ota-android-agent)

Effort: L **Source:** Accounts delivery plan M6, `docs/research/accounts/25_device_side_update_client.md`

What to build:

- 1. Setup wizard UI:**
 - Welcome screen
 - Server URL configuration (or QR code scan)
 - Device identity display (hardware ID, model, OS version)
 - Account selection / pairing
 - Consent screen (data collection, update policy)
 - Completion screen

2. Notification channel:

- Update available notification
- Download progress notification
- Install ready notification (reboot required)
- Post-install success/failure notification

3. Integration with PollScheduler:

- Wizard triggers initial registration
- After setup, PollScheduler takes over periodic checks

Dependency on M7/C-07: Full e2e (download + apply + reboot) requires the object-storage seam so the device can actually download artifacts.

Verification Checklist

Step	Action	Expected Result
E-01	Board attached and reachable	adb devices shows device
E-02a	Control-plane reachable	Device pings ota-server
E-02b	Registration	Device receives token
E-02c	Update check	Server returns update offer
E-02d	Download + verify	SHA-256 + Ed25519 pass
E-02e	Apply via update_engine	Payload written to inactive slot
E-02f	Reboot + verify	New slot boots, version matches
E-02g	Telemetry	Server receives success event
E-03	ApplyPort full loop	Go ApplyPort completes Steps a → f
E-04	Android stress/chaos/bench	All test types pass, no leaks
E-05	Bridge real-device tests	update_engine callbacks work
E-06	Setup wizard	End-to-end registration flow

Honest Boundary (§11.4.6)

- E-01 and E-02 require hardware the agent cannot provide. They are OPERATOR-BLOCKED (OTA-004 in Issues.md).

- E-03 (ApplyPort) is currently a documented SCAFFOLD. It was read this session — the scaffold header at `applyport.go:10-16` is accurate.
- E-04 and E-05 (Android tests) are partially blocked on hardware. JVM unit tests can be added without hardware.
- E-06 (setup wizard) is design-only (referenced in Accounts plan, not yet started). It can begin without hardware.